Mohamed bin Zayed University of Artificial Intelligence

Software Architecture Diagramming

Reducing ambiguity • Aligning teams • Preventing costly mistakes

Why Visual Models Matter

Mohamed bin Zayed University of Artificial Intelligence

Reduce Ambiguity

Complex systems easier to reason about when structure is visible

Make Trade-offs Explicit

Helps teams choose the right architecture sooner

Expose Hidden Dependencies

Preventing late surprises in implementation and testing

Speed Onboarding

New team members grasp the system faster

Align Stakeholders

Reducing rework caused by conflicting mental models

Save Time Overall

Minutes of diagramming prevents weeks of refactoring

Agenda

1. **Modern UML** — Shared notation for teams
2. **C4 Model** — Architecture alternative
3. **Four C4 Levels** — Different zoom levels
4. **Mermaid** — Diagrams close to code
5. **Compare Types** — Right diagram for the job
6. **Lessons Learned** — Apply to your projects

Agenda

Mohamed bin Zayed University of Artificial Intelligence

From “why diagram?” to concrete tools and examples

- Why diagramming matters (communication, reasoning, risk reduction)
- What architecture diagrams should represent (and what they should omit)
- UML theory essentials: structural vs behavioral views
- The C4 model: Context → Container → Component → Code
- Diagrams-as-code: versioning, reviews, automation
- Mermaid: practical examples you can paste into docs today

Guiding idea

A diagram is a *view* of a system.
The “right” diagram is the one that answers a stakeholder’s question with minimal ambiguity.

Deliverables

- A repeatable approach
- Example diagrams (C4 + UML)
- Mermaid snippets
- Tool selection cheat-sheet

UML: The Heritage

Understanding where we came from

Mohamed bin Zayed University of Artificial Intelligence

EVOLUTION

- **1990s** — Object-oriented methods proliferate
- **1997** — UML 1.0 adopted by OMG
- **2005** — UML 2.0 with 14 diagram types
- **Today** — Limited adoption in modern teams

14 UML DIAGRAM TYPES

STRUCTURAL (7)

Class • Object • Component
Package • Deployment
Composite Structure • Profile

BEHAVIORAL (7)

Use Case • Activity • State
Sequence • Communication
Interaction • Timing

Key Insight: Most teams use only 3-4 diagram types. Class, Sequence, and Component diagrams remain the most practical.

UML Diagram Types

A shared grammar that prevents costly miscommunication

Mohamed bin Zayed University of Artificial Intelligence

Structure Diagrams

Class Diagrams
Data models, inheritance, relationships

Component Diagrams
Modular boundaries, dependencies

Deployment Diagrams
Infrastructure mapping, topology

Behavior Diagrams

Sequence Diagrams
Message order, interaction flows

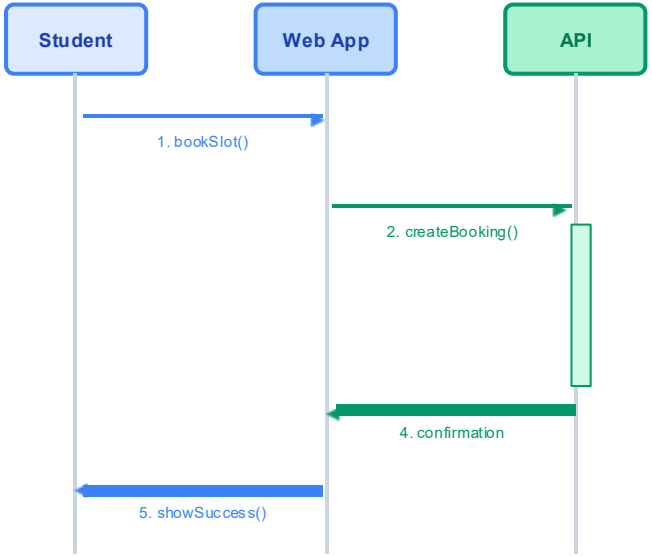
Activity Diagrams
Workflows, decision points

State Diagrams
Lifecycle control, transitions

Standard symbols help teams document just enough without inventing notation

Sequence Diagrams

Show message order to prevent race conditions and timing bugs



Clarify responsibilities

Avoid duplicated logic across services

Reveal missing error handling

Reduces production incidents

Document integration contracts

Helps teams coordinate changes

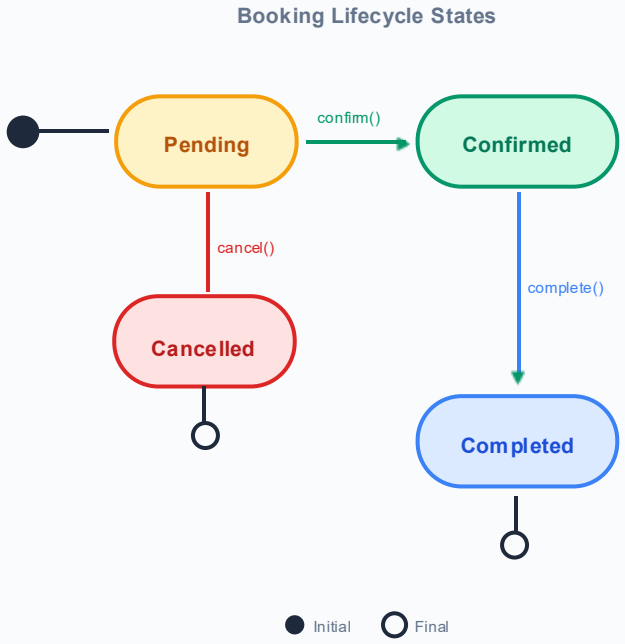
Make async flows understandable

Saves significant debug time

State Diagrams

Prevent invalid transitions and reduce logic errors

Mohamed bin Zayed University of Artificial Intelligence



Crucial for reactive systems

Where order and timing matter

Make edge cases visible

Improving system reliability

Guide test case design

Each transition is a test scenario

Clear user-facing behavior

Reduces ambiguous feature specs

UML Challenges in Modern Software

Mohamed bin Zayed University of Artificial Intelligence

Too Complex

14 diagram types with extensive notation. High learning curve discourages adoption.

Wrong Abstraction Level

Focuses on code-level details (classes, methods) rather than system architecture.

Tooling Friction

Expensive tools, proprietary formats, and diagrams that don't live with code.

Maintenance Burden

Diagrams quickly become outdated. No versioning or automated synchronization.

The Result: Teams either abandon diagrams entirely, or create inconsistent "box and arrow" drawings with no shared vocabulary.

Why Multiple Diagramming Lenses?

No single diagram answers every question

Mohamed bin Zayed University of Artificial Intelligence

UML

Unified Modeling Language

Best For:

- Detailed structure and behavior
- Design contracts and reviews
- Precision documentation

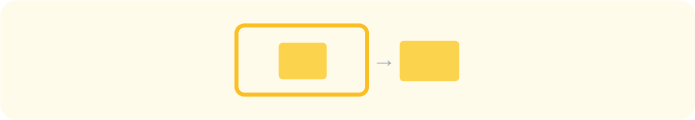
+

C4 Model

Context, Container, Component, Code

Best For:

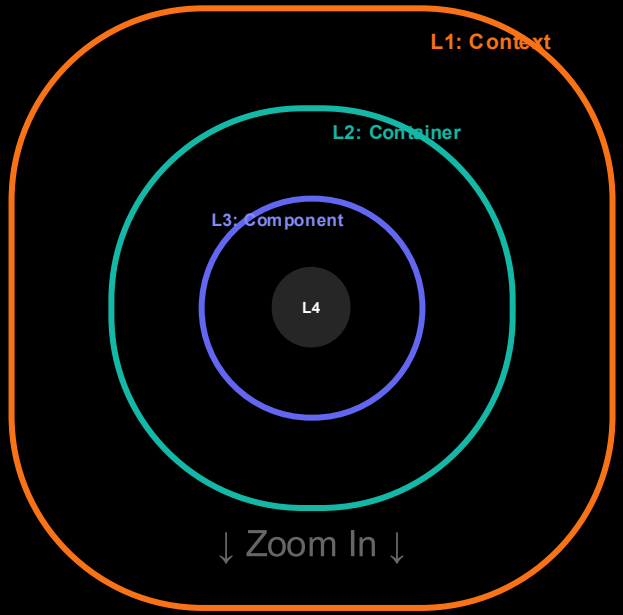
- Architectural zoom levels
- Stakeholder communication
- Zoomable storytelling



Using both avoids gaps between big-picture and code-level decisions

What Is the C4 Model?

A lightweight way to describe software architecture



Four Zoom Levels

Matches how engineers reason about systems

Clarity Over Completeness

Keeps documentation usable and relevant

Designed for Communication

Stakeholders can follow the story

Agile-Friendly

Easy to update as systems evolve

Choose diagram types by the question

Mohamed bin Zayed University of Artificial Intelligence

Different views answer different stakeholder concerns

Context

What is the system?
Who uses it?
What does it depend on?

Structure

What are the big building blocks?
How are responsibilities split?

Behavior

What happens at runtime?
What talks to what, and when?

Deployment

Where does it run?
What are the zones, nodes, and connections?

Data

What's stored where?
What are the core entities and flows?

Tip: A "diagram set" is stronger than a single mega-diagram.

Progressive disclosure

Start high-level.
Zoom in only where decisions or risks demand it.

Audience and viewpoints

Architecture descriptions organize views to address stakeholder concerns

Stakeholder → Typical questions

Stakeholder	Concerns the diagram must answer
Product / Business	What capabilities exist? What are the external dependencies?
Engineers	What are the modules, APIs, and data contracts?
Ops / SRE	Where does it run? How does traffic flow? Where do we observe?
Security	What crosses trust boundaries? Where is sensitive data stored?

Key implication

A single diagram rarely satisfies everyone.
Use a *set* of diagrams, each with a clear viewpoint and audience.

Standards perspective

ISO/IEC/IEEE 42010 emphasizes stakeholder concerns, viewpoints, and views as the structure of an architecture description.

C4 Level 1: System Context (example)

Show the system in scope and its relationships

Reading (and drawing) context diagrams

Mohamed bin Zayed University of Artificial Intelligence

Make boundaries, dependencies, and users unmistakable

Include:

- Named system boundary
- Primary user roles/personas
- External systems you integrate with
- Labeled relationships (verb + protocol where helpful)
- A legend for any special notation

Avoid:

- Internal code structure (too detailed)
- Every microservice in the company
- Network topology (belongs in deployment)
- Arrows without meaning

Good context diagrams are “product-like”

They work for technical and non-technical stakeholders: the nouns are familiar (people + systems) and the verbs explain interactions.

Suggested cadence

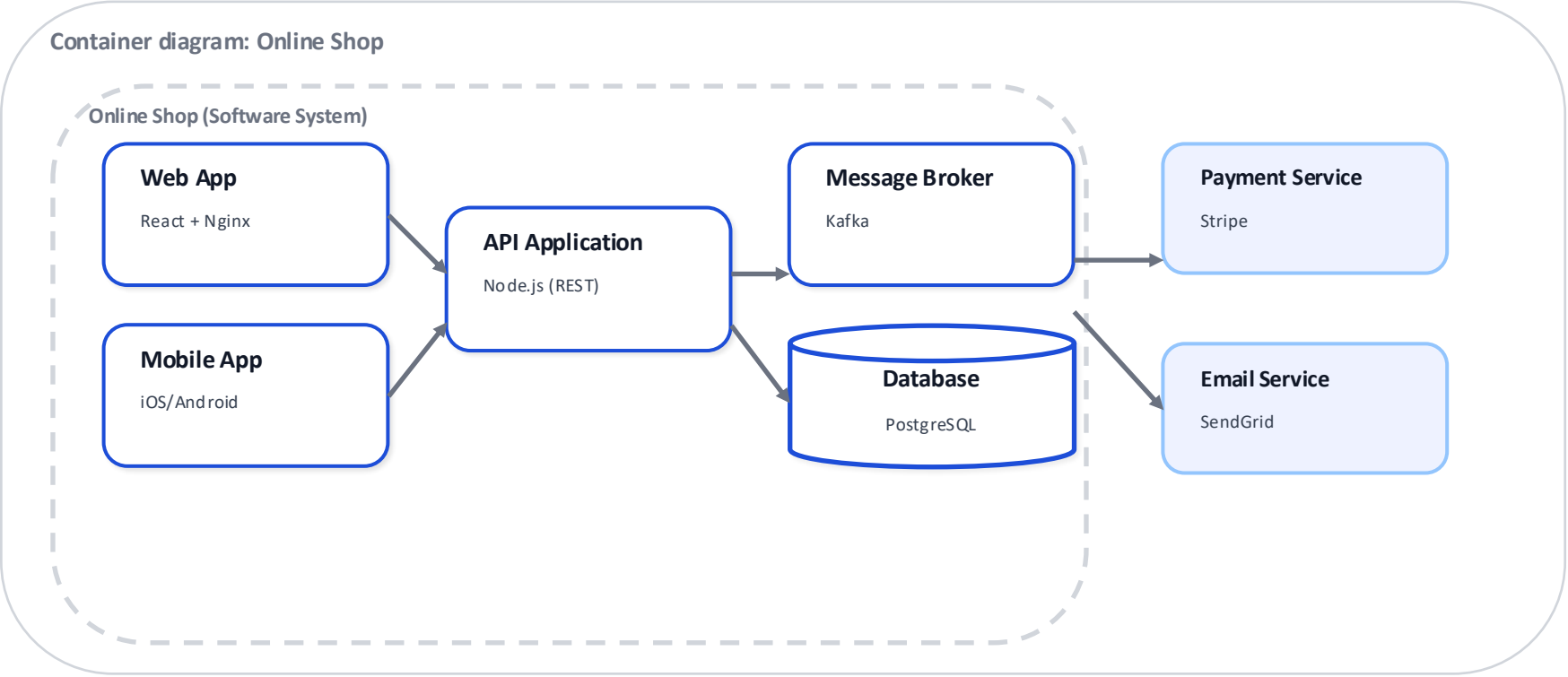
Update when dependencies change.
Review during architecture decision points (new integration, new trust boundary, major refactor).

Minimum metadata

Title • Owner • Last updated • Audience
(These small details keep diagrams usable over time.)

C4 Level 2: Container diagram (example)

Open the “black box” to show runnable/deployable building blocks



Container diagrams: what to capture

Responsibilities, technologies, and key runtime interactions

Responsibility

- What does it do?
- What does it own?

Technology choices

- Language/framework
- Data store type
- Protocols

Runtime qualities

- SLO targets
- Scaling model
- Failure modes

Mini-example: API Application container

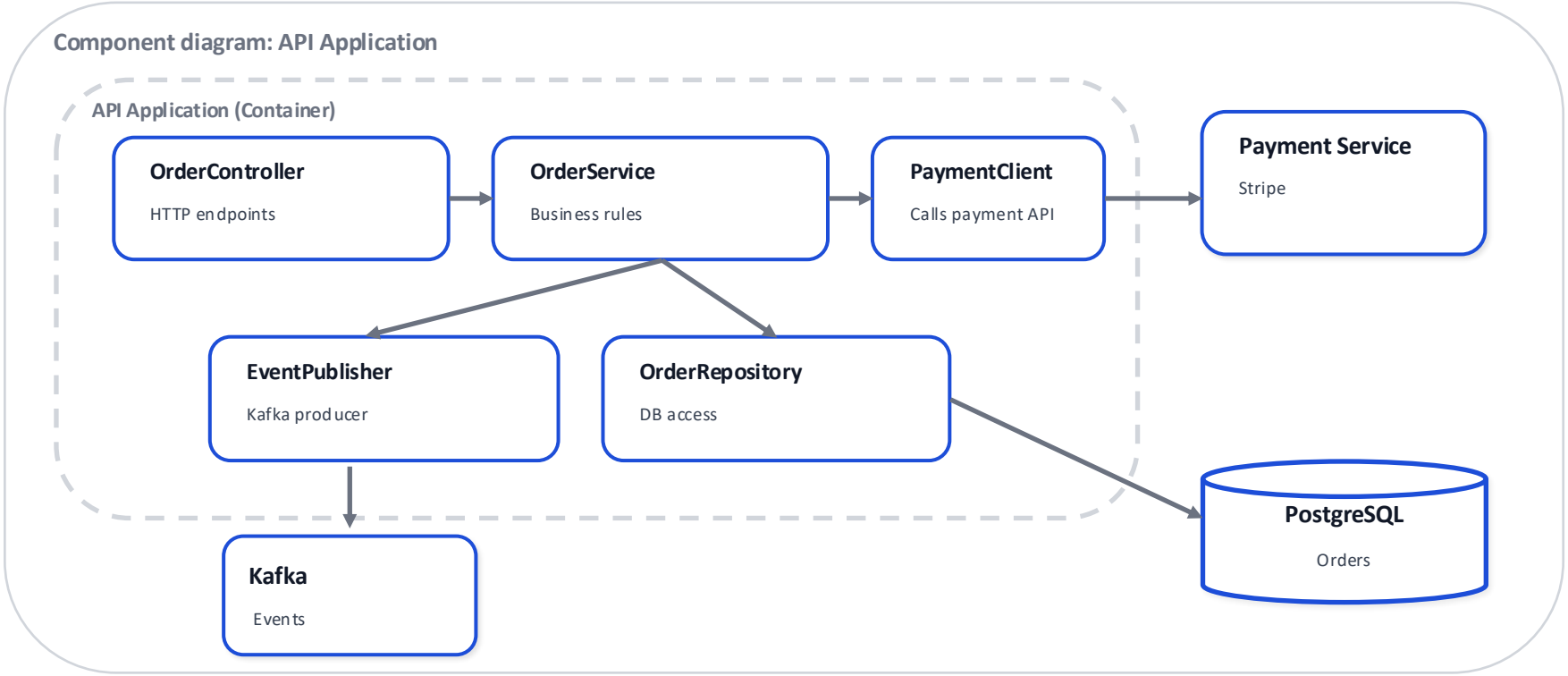
API Application (Node.js)

- Provides REST endpoints: /catalog, /orders
- Validates requests, orchestrates workflows
- Publishes events to Kafka
- Reads/writes PostgreSQL

Add: trust boundary notes (authn/authz), retry/timeout policies, and key metrics (latency, error rate).

C4 Level 3: Component diagram (example)

Zoom into one container and show the major building blocks



Component in C4

A component is a grouping of related functionality encapsulated behind a well-defined interface (not separately deployable).

C4 Level 4: Code (optional)

Go here only when you need “street-level” detail

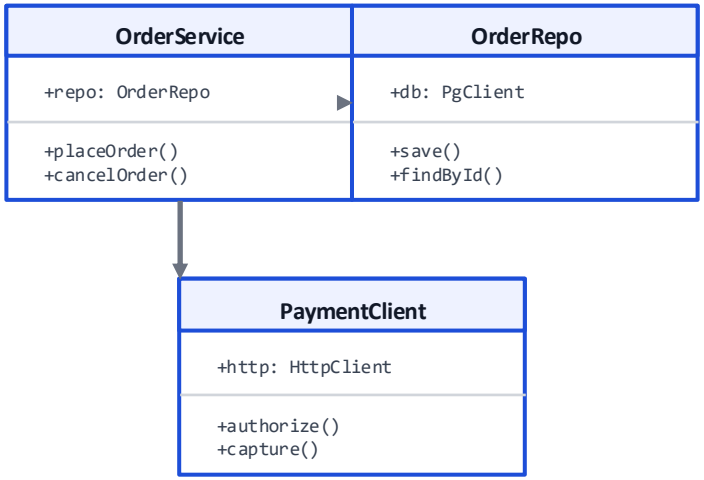
What Level 4 is

A closer look at implementation: classes, interfaces, modules, functions. Often produced automatically by IDEs or expressed in UML class diagrams.

When to stop zooming

If the diagram isn’t helping with a decision, risk, or onboarding goal — don’t add detail.

Mini example (UML class style)



Tip: generate these automatically (or keep them near the code).

Using C4 and UML together

C4 for structure + navigation; UML for precision

Mohamed bin Zayed University of Artificial Intelligence

A pragmatic pairing:

C4 answers “where are we?”

- Context / container / component navigation
- Boundaries and ownership
- Key integrations and dependencies

UML answers “what happens / how exactly?”

- Sequence diagrams for critical flows
- State machines for complex lifecycle logic
- Deployment diagrams for infrastructure detail

Workflow suggestion

Start with C4 Level 1–2 for shared understanding.
Add Level 3 or UML sequence diagrams only for the parts you actively design or change.

Why Diagram as Code?

Treating diagrams like software artifacts

Mohamed bin Zayed University of Artificial Intelligence

TRADITIONAL DIAGRAMMING PAIN

Binary Files

Visio, Draw.io, Lucidchart files can't be meaningfully diffed or merged

No Version Control

Who changed what? When? Why? History is lost

Tool Lock-in

Expensive licenses, proprietary formats, collaboration friction

Separate from Code

Documentation drifts from reality

DIAGRAM AS CODE BENEFITS

Version Controlled

Store in Git alongside code. Full history, branching, PRs for diagram changes

Text-Based Diffs

See exactly what changed in each commit. Meaningful code reviews

CI/CD Integration

Auto-generate diagrams on build. Embed in docs. Always current

No Special Tools

Edit in any text editor. Render anywhere. Open formats

A diagrams-as-code workflow

Make it easy to update, render, and publish

Mohamed bin Zayed University of Artificial Intelligence

1 Write

Text/DSL near the code (mermaid, PlantUML, Structurizr, etc.)

2 Review

PR review like any other change
(changes are diffable)

3 Render

CI renders to SVG/PNG/PDF (or platforms render automatically)

4 Publish

Docs site / README / wiki kept in sync with releases

Two easy wins

1) Put the source next to the code it describes.
2) Render diagrams automatically (CI or your docs platform).

Diagram-as-Code Tools

Mohamed bin Zayed University of Artificial Intelligence

Mermaid

JavaScript-based, renders in browsers. Native GitHub/GitLab support.

PlantUML

Java-based, mature ecosystem. Extensive UML support, C4 extensions.

Structurizr

DSL specifically for C4. From Simon Brown. Workspace concept.

Feature	Mermaid	PlantUML	Structurizr
Learning Curve	Easy ⭐	Medium	Medium
GitHub Native	✓ Yes	Plugin	Export
C4 Support	Extension	Extension	Native ⭐

Today's focus: Mermaid – the most accessible option with growing ecosystem

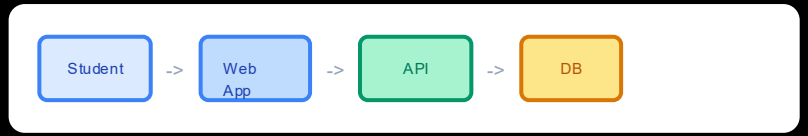
Mermaid: Diagrams as Code

Keep diagrams in the same repo as your code

MERMAID SYNTAX

```
flowchart LR
  A[Student] --> B[Web App]
  B --> C[API Service]
  C --> D[(Database)]
  C --> E[Email Service]
```

RENDERS TO



Plain Text Format

Lives in the same repo as code

Diffable Changes

Design changes are reviewable in PRs

Markdown Integration

Easy to publish and share docs

Reduces Drift

Diagrams close to code stay current

Choosing the Right Diagram

Match the diagram type to the engineering question

QUESTION	DIAGRAM TYPE	WHAT IT REVEALS
Who uses the system?	C4 Context	Users, external systems, trust boundaries
What are the major parts?	C4 Container	Runtime components, data flows, tech choices
How is work divided?	C4 Component	Internal structure, team ownership
What does the data look like?	UML Class	Entity relationships, inheritance, APIs
In what order do things happen?	UML Sequence	Message flow, timing, error paths
What states can it be in?	UML State	Lifecycle, valid transitions, edge cases
Where does it run?	UML Deployment	Infrastructure, network topology, failure points

Best Practices

Mohamed bin Zayed University of Artificial Intelligence

Content Guidelines

- **Title every diagram** — what and scope
- **Label all elements** — name + technology
- **Describe relationships** — what flows
- **Add a legend** — explain notation
- **Include date/version** — when updated

Maintenance

- **Store with code** — same repo
- **Review in PRs** — architecture changes
- **Automate rendering** — CI/CD pipeline
- **Delete stale diagrams** — outdated is useless
- **Assign ownership** — who updates?

Audience Focus

- **Know your reader** — tech level?
- **One diagram, one purpose**
- **5-7 elements max** — cognitive limit
- **Zoom only when needed**

Common Pitfalls

- Too much detail — can't see the forest
- Missing context — floating boxes
- Inconsistent notation — confusion
- Never updating — fiction not docs

Common Diagramming Anti-Patterns

Pitfalls that reduce diagram value

Mohamed bin Zayed University of Artificial Intelligence

Overly Detailed

Wastes time and obscures key decisions

Stale Diagrams

Erodes trust, teams ignore documentation

Missing Context

Leads to wrong interpretations and bad decisions

One-Size-Fits-All

Fails to answer specific engineering questions

Inconsistent Notation

Confuses readers, reduces diagram value

Diagramming Too Late

Becomes documentation, not a design tool

Solution: Diagram early, iterate often, keep them close to code with Mermaid

Key Takeaways

Mohamed bin Zayed University of Artificial Intelligence

Architecture diagrams are essential communication tools

They bridge the gap between technical and non-technical stakeholders

C4 provides the right levels of abstraction

Context → Container → Component → Code: zoom only when needed

Diagram-as-code enables maintainable documentation

Version control, code review, and CI/CD for diagrams just like code

Mermaid is the easiest entry point

Native GitHub support, simple syntax, no setup required

Questions?

C4 Model	Mermaid	Live Editor	Structurizr
c4model.com	mermaid.js.org	mermaid.live	structurizr.com

Thank you for your attention!